

Exercise 1: Create a Spring Web Project using Maven

A Spring Boot project named spring-learn was created using Spring Initializr with the required dependencies. The project was imported into Eclipse as a Maven project, and the application was executed successfully. The console confirmed that the Spring Boot application started without any errors.

Source Code

[SpringLearnApplication.java](#)

```
package com.cognizant.springlearn;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringLearnApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
    }
}
```

Figure 1: Spring Initializr project configuration.

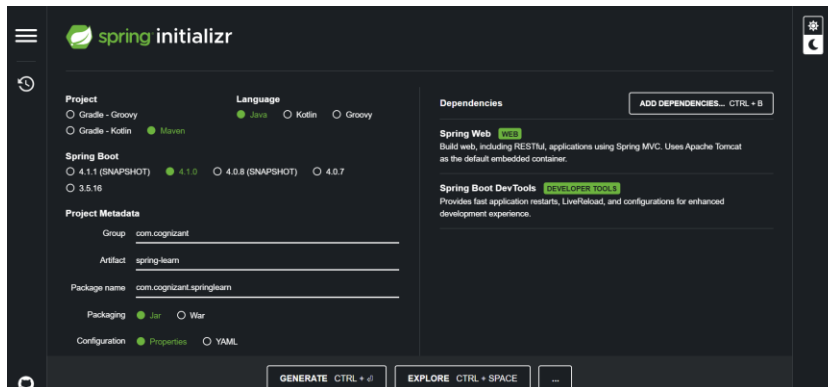


Figure 2: Project imported successfully in Eclipse.

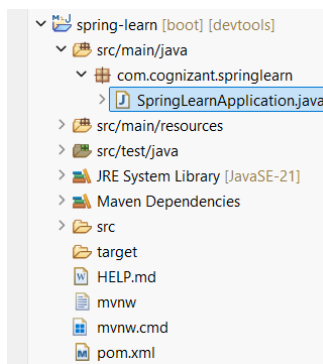


Figure 3: Console showing successful execution of SpringLearnApplication.



Exercise 2: Spring Core – Load Country from Spring Configuration XML

In this exercise, the Country bean was configured using a Spring XML configuration file. The country.xml file was created under the resources folder, and the bean properties such as country code and country name were defined. The Country class and SpringLearnApplication were updated to load the XML configuration using the Spring IoC container. After running the application, the configured Country object was successfully loaded and displayed in the console.

Source Code

[country.xml](#)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
  </bean>
</beans>
```

```
</bean>
</beans>
```

Country.java

```
package com.cognizant.springlearn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class Country {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(Country.class);
    private String code;
    private String name;
    public Country() {
        LOGGER.debug("Inside Country Constructor.");
    }
    public String getCode() {
        LOGGER.debug("Inside getCode()");
        return code;
    }
    public void setCode(String code) {
        LOGGER.debug("Inside setCode()");
        this.code = code;
    }
    public String getName() {
        LOGGER.debug("Inside getName()");
        return name;
    }
    public void setName(String name) {
        LOGGER.debug("Inside setName()");
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

SpringLearnApplication.java

```
package com.cognizant.springlearn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
@SpringBootApplication
public class SpringLearnApplication {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);
    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
        displayCountry();
    }
    public static void displayCountry() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.debug("Country : {}", country.toString());
        LOGGER.info("END");
    }
}
```

Figure 1: country.xml configuration file.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <beans xmlns="http://www.springframework.org/schema/beans"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5       xsi:schemaLocation="
6         http://www.springframework.org/schema/beans
7         https://www.springframework.org/schema/beans/spring-beans.xsd">
8
9     <bean id="country" class="com.cognizant.springlearn.Country">
10         <property name="code" value="IN"/>
11         <property name="name" value="India"/>
12     </bean>
13
14 </beans>
```

Figure 2: Country.java.

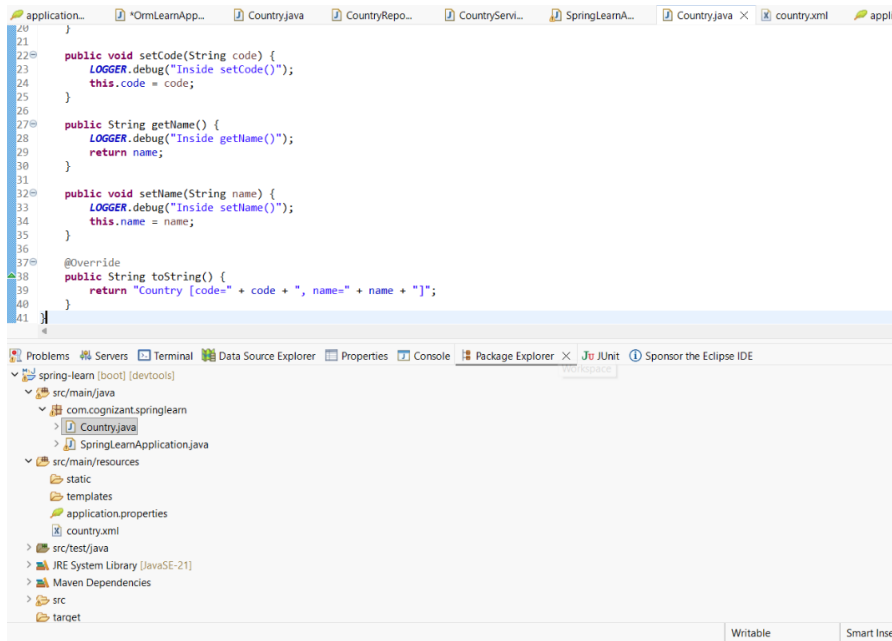


Figure 3: SpringLearnApplication.java.

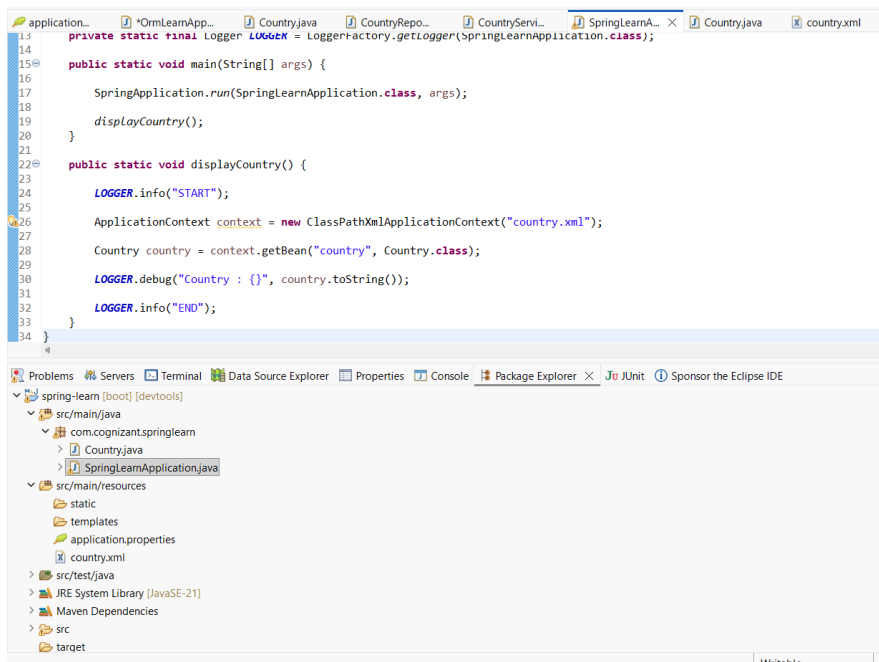
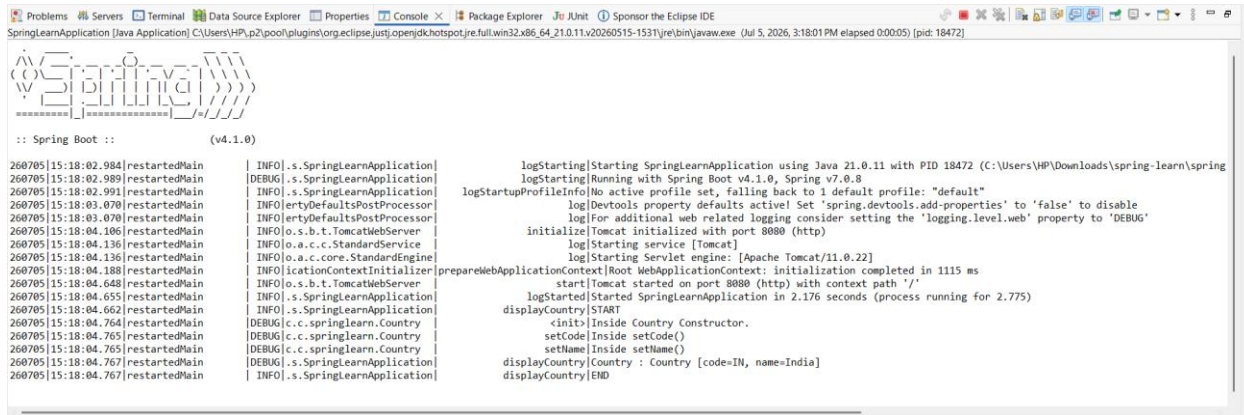


Figure 4: Console showing successful loading of the Country bean from the Spring XML configuration.



```
SpringLearnApplication [Java Application] C:\Users\HPA\p2\pool\plugins\org.eclipse.just\openjdk.hotspot.jre.full.win32.x86_64.21.0.11.v20260515-1531\jre\bin\javaw.exe (Jul 5, 2026, 3:18:01 PM elapsed 0:00:05) [pid: 18472]

:: Spring Boot :: (v4.1.0)

260785|15:18:02.988|restartMain|INFO|.s.SpringLearnApplication|logStarting|Starting SpringLearnApplication using Java 21.0.11 with PID 18472 (C:\Users\HPA\p2\pool\plugins\org.eclipse.just\openjdk.hotspot.jre.full.win32.x86_64.21.0.11.v20260515-1531\jre\bin\javaw.exe)
260785|15:18:02.989|restartMain|DEBUG|.s.SpringLearnApplication|logStarting|Running with Spring Boot v4.1.0, Spring v7.0.8
260785|15:18:02.991|restartMain|INFO|.s.SpringLearnApplication|logStartupProfileInfo|No active profile set, falling back to 1 default profile: "default"
260785|15:18:03.070|restartMain|INFO|ertyDefaultsPostProcessor|log|Devtools property defaults active! Set 'spring.devtools.add-properties' to 'false' to disable
260785|15:18:04.106|restartMain|INFO|o.s.b.t.TomcatWebServer|log|For additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
260785|15:18:04.136|restartMain|INFO|o.a.c.c.StandardService|initialize|Tomcat initialized with port 8080 (http)
260785|15:18:04.136|restartMain|INFO|o.a.c.c.StandardEngine|log|Starting service [Tomcat]
260785|15:18:04.188|restartMain|INFO|icationContextInitializer|log|Starting Servlet engine: [Apache Tomcat/11.0.22]
260785|15:18:04.648|restartMain|INFO|o.s.b.t.TomcatWebServer|prepareWebApplicationContext|Root WebApplicationContext: initialization completed in 1115 ms
260785|15:18:04.655|restartMain|INFO|.s.SpringLearnApplication|start|Tomcat started on port 8080 (http) with context path '/'
260785|15:18:04.662|restartMain|INFO|.s.SpringLearnApplication|logStarted|Started SpringLearnApplication in 2.176 seconds (process running for 2.775)
260785|15:18:04.764|restartMain|DEBUG|c.c.springlearn.Country|displayCountry|START
260785|15:18:04.765|restartMain|DEBUG|c.c.springlearn.Country|<init>|Inside Country Constructor.
260785|15:18:04.765|restartMain|DEBUG|c.c.springlearn.Country|setCode|Inside setCode()
260785|15:18:04.765|restartMain|DEBUG|c.c.springlearn.Country|setName|Inside setName()
260785|15:18:04.767|restartMain|DEBUG|.s.SpringLearnApplication|displayCountry|Country : Country [code=IN, name=India]
260785|15:18:04.767|restartMain|INFO|.s.SpringLearnApplication|displayCountry|END
```

Exercise 3: Hello World RESTful Web Service

A RESTful web service was created using Spring Boot. A `HelloController` class was developed with the `@RestController` annotation, and a GET endpoint `/hello` was created using `@GetMapping`. When the application was executed and the URL `http://localhost:8080/hello` was accessed in the browser, the service returned the message **"Hello World!!"**, confirming that the REST API was working successfully.

Source Code

HelloController.java

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);

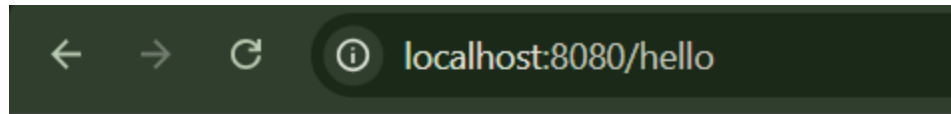
    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("START");
        String message = "Hello World!!";
        LOGGER.info("END");
        return message;
    }
}
```

```

    }
}

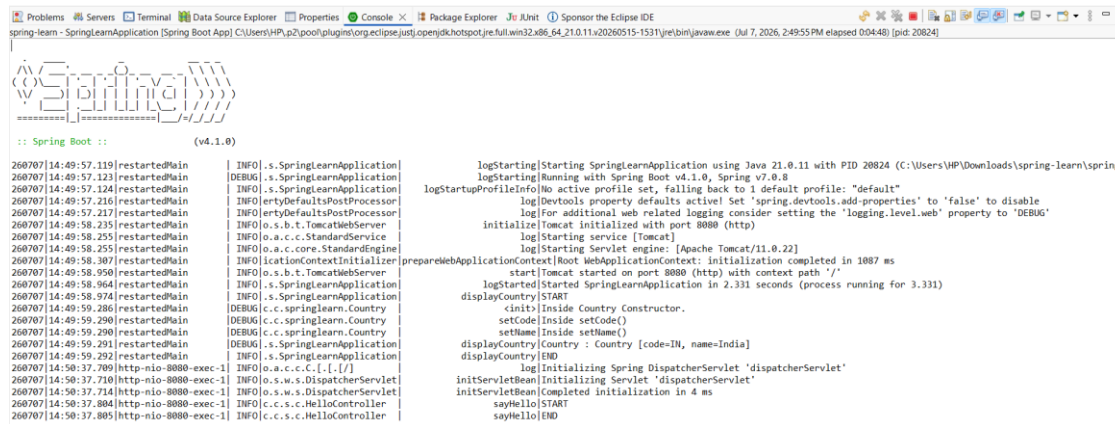
```

Figure 1:



Hello World!!

Figure 2:



Exercise 4: REST – Country Web Service

A RESTful web service was developed to return the details of a country using Spring Boot. A CountryService class was created to load the Country bean from the country.xml configuration file. A CountryController class was developed with a GET endpoint /country that calls the service and returns the Country object. When the application was executed and the URL <http://localhost:8080/country> was accessed in the browser, the country details were successfully returned in JSON format.

Source Code

CountryService.java

```

package com.cognizant.springlearn.service;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

```

```

import org.springframework.stereotype.Service;
import com.cognizant.springlearn.Country;
@Service
public class CountryService {
    public Country getCountry() {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        return context.getBean("country", Country.class);
    }
}

```

CountryController.java

```

package com.cognizant.springlearn.controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;
@RestController
public class CountryController {
    @Autowired
    private CountryService countryService;
    @GetMapping("/country")
    public Country getCountry() {
        return countryService.getCountry();
    }
}

```

Figure 1: Browser showing the output of <http://localhost:8080/country>.

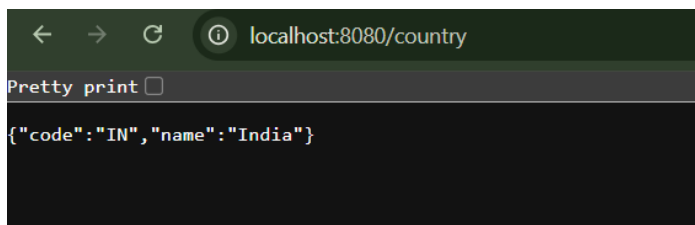
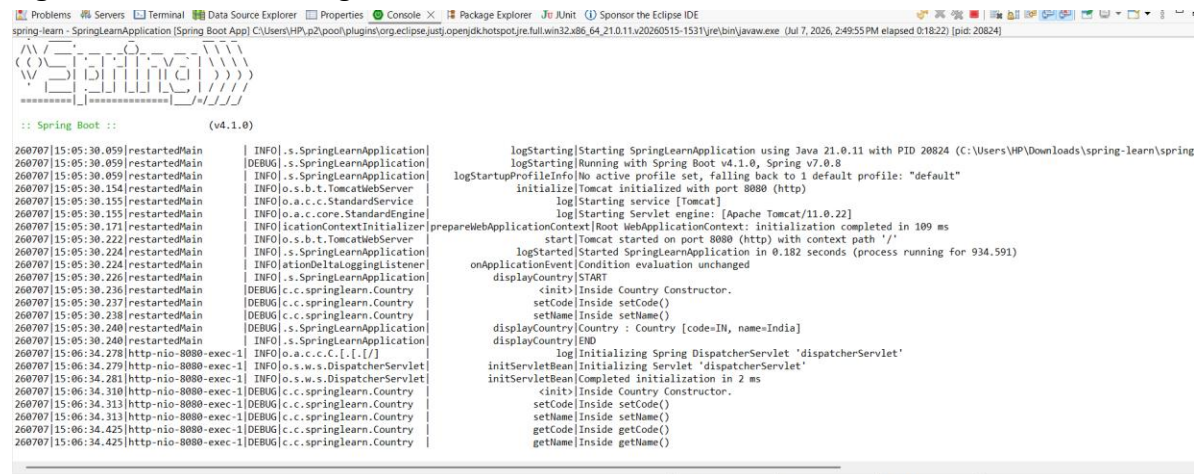


Figure 2: Console showing successful execution of the REST service.



```
spring-learn - SpringLearnApplication [Spring Boot App] C:\Users\HP\AppData\Local\Temp\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20200515-1531\jre\bin\java.exe (Jul 7, 2026, 2:49:55 PM elapsed 0:18:22) [pid: 20824]

:: Spring Boot :: (v4.1.0)

260707|15:05:30.059|restartedMain|INFO|.s.SpringLearnApplication|logStarting|Starting SpringLearnApplication using Java 21.0.11 with PID 20824 (C:\Users\HP\Downloads\spring-learn\spring
260707|15:05:30.059|restartedMain|DEBUG|.s.SpringLearnApplication|logStarting|Running with Spring Boot v4.1.0, Spring v7.0.8
260707|15:05:30.059|restartedMain|INFO|.s.SpringLearnApplication|logStartupProfileInfo|No active profile set, falling back to 1 default profile: "default"
260707|15:05:30.154|restartedMain|INFO|.s.b.t.TomcatWebServer|initialize|Tomcat initialized with port 8080 (http)
260707|15:05:30.155|restartedMain|INFO|.a.c.c.StandardService|log|Starting service [Tomcat]
260707|15:05:30.155|restartedMain|INFO|.a.c.c.core.StandardEngine|log|Starting Servlet engine: [Apache Tomcat/11.0.22]
260707|15:05:30.171|restartedMain|INFO|.s.b.t.TomcatWebServer|start|Tomcat started on port 8080 (http) with context path '/'
260707|15:05:30.224|restartedMain|INFO|.s.SpringLearnApplication|log|Started Started SpringLearnApplication in 0.182 seconds (process running for 934.591)
260707|15:05:30.224|restartedMain|INFO|.s.SpringLearnApplication|onApplicationEvent|Condition evaluation unchanged
260707|15:05:30.226|restartedMain|INFO|.s.SpringLearnApplication|displayCountry|START
260707|15:05:30.236|restartedMain|DEBUG|.c.c.springlearn.Country|<init>|Inside Country Constructor.
260707|15:05:30.237|restartedMain|DEBUG|.c.c.springlearn.Country|setCode|Inside setCode()
260707|15:05:30.238|restartedMain|DEBUG|.c.c.springlearn.Country|setName|Inside setName()
260707|15:05:30.240|restartedMain|DEBUG|.s.SpringLearnApplication|displayCountry|Country : Country [code=IN, name=India]
260707|15:05:30.240|restartedMain|INFO|.s.SpringLearnApplication|displayCountry|END
260707|15:06:34.278|http-nio-8080-exec-1|INFO|.a.c.c.c.c.[./|log|Initializing Spring DispatcherServlet 'dispatcherServlet'
260707|15:06:34.279|http-nio-8080-exec-1|INFO|.p.s.w.s.DispatcherServlet|initServletBean|Initializing Servlet 'dispatcherServlet'
260707|15:06:34.281|http-nio-8080-exec-1|INFO|.p.s.w.s.DispatcherServlet|initServletBean|Completed initialization in 2 ms
260707|15:06:34.310|http-nio-8080-exec-1|DEBUG|.c.c.springlearn.Country|<init>|Inside Country Constructor.
260707|15:06:34.313|http-nio-8080-exec-1|DEBUG|.c.c.springlearn.Country|setCode|Inside setCode()
260707|15:06:34.313|http-nio-8080-exec-1|DEBUG|.c.c.springlearn.Country|setName|Inside setName()
260707|15:06:34.425|http-nio-8080-exec-1|DEBUG|.c.c.springlearn.Country|getCode|Inside getCode()
260707|15:06:34.425|http-nio-8080-exec-1|DEBUG|.c.c.springlearn.Country|getName|Inside getName()
```

Exercise 5: REST – Get Country based on Country Code

A RESTful web service was developed to return the details of a country based on the country code provided in the URL. The CountryService class was updated to search for the country using the given code, and the CountryController class was modified to accept the country code as a path variable. When the application was executed and the URL `http://localhost:8080/country/IN` was accessed, the service successfully returned the country details in JSON format.

Source Code

CountryService.java

```
package com.cognizant.springlearn.service;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;
import com.cognizant.springlearn.Country;

@Service
public class CountryService {

    public Country getCountry(String code) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        if (country.getCode().equalsIgnoreCase(code)) {
            return country;
        }
        return null;
    }
}
```

```
}  
}
```

CountryController.java

```
package com.cognizant.springlearn.controller;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.RestController;  
import com.cognizant.springlearn.Country;  
import com.cognizant.springlearn.service.CountryService;  
@RestController  
public class CountryController {  
    @Autowired  
    private CountryService countryService;  
    @GetMapping("/country/{code}")  
    public Country getCountry(@PathVariable String code) {  
        return countryService.getCountry(code);  
    }  
}
```

Figure 1: Browser showing the output of <http://localhost:8080/country/IN>.

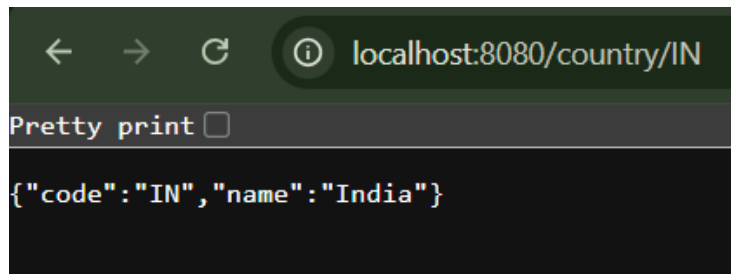
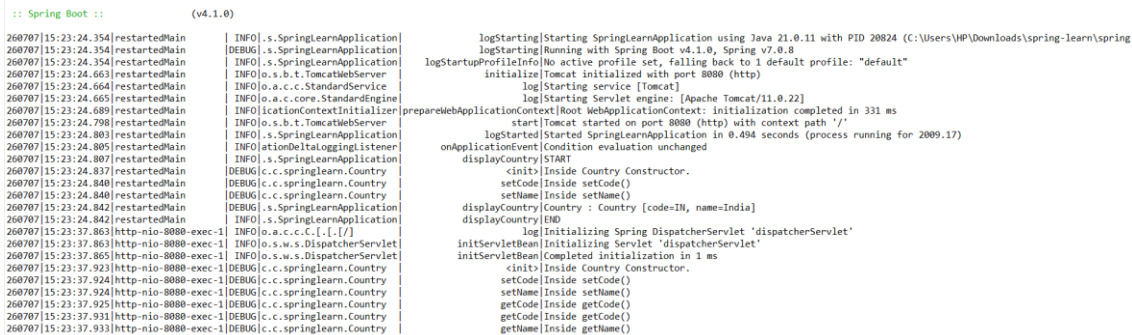


Figure 2: Console showing successful execution of the application.



The RESTful web service successfully accepted the country code through the URL and returned the corresponding country details in JSON format.

In this exercise, a REST controller was created to provide an authentication endpoint. The /authenticate endpoint was implemented and tested successfully. When the endpoint was accessed through the browser, the application returned the response **"JWT Token Generated"**, confirming that the authentication service was working correctly.

AuthenticationController.java

```
package com.cognizant.springlearn.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

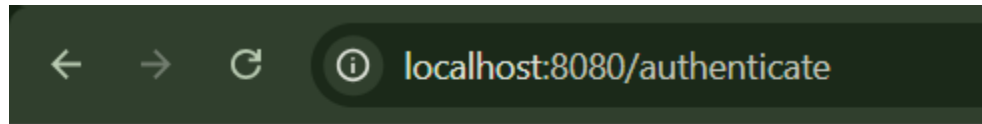
@RestController

public class AuthenticationController {
```

```
@GetMapping("/authenticate")
public String authenticate() {
    return "JWT Token Generated";
}
```

Figure 1: AuthenticationController.java.

Figure 2:



JWT Token Generated